

BÁO CÁO

Họ & Tên: Vũ Quốc Hùng	MSSV: 24207015
Lớp: 24DTV-DKD1	Ca: 15-17h

Bài 01 ÔN TẬP KỸ THUẬT LẬP TRÌNH

- Đọc ghi file:
 - Đọc một danh sách các số nguyên từ một file văn bản.
 - Lưu các số nguyên này vào một mảng sử dụng con trỏ.
 - Sắp xếp mảng theo thứ tự tăng dần.
 - Ghi mảng đã được sắp xếp vào một file khác.
 - In mảng đã được sắp xếp ra màn hình.

```
#include <stdio.h>
#include <stdlib.h>
#define N 100
void xuấtmang(int *pa, int len) {
    for (int i = 0; i < len; i++)
        printf("%d ", *(pa + i));
    printf("\n");
}
void bubblesort(int *pa, int n) {
    int temp;
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (*(pa + i) > *(pa + j)) {
                temp = *(pa + i);
                *(pa + i) = *(pa + j);
                *(pa + j) = temp;
            }
        }
    }
}
int docfile(char *tenFile, int *pa) {
    FILE *fi = fopen(tenFile, "rt");
    int count = 0, temp;
    while (!feof(fi) && count < N) {
        if (fscanf(fi, "%d", &temp) == 1) {
            *(pa + count) = temp;
            count++;
        }
    }
    fclose(fi);
    return count;
}
void ghifile(char *tenFile, int *pa, int n) {
    FILE *fo = fopen(tenFile, "wt");
    for (int i = 0; i < n; i++)
```

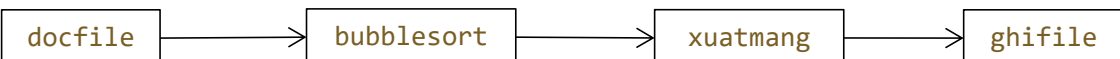
```

    fprintf(fo, "%d ", *(pa + i));
    fclose(fo);
}
int main() {
    int a[N];
    int n = docfile("input.txt", a);
    printf("mang truoc khi sap xep:");
    xuatmang(a, n);
    bubblesort(a, n);
    printf("mang sau khi sap xep:");
    xuatmang(a, n);
    ghifile("output.txt", a, n);
    return 0;
}

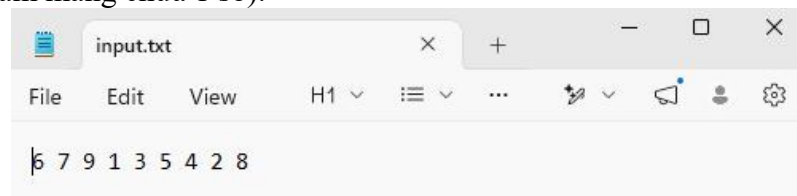
```

Giải thích cách hoạt động của code:

1. Hàm **xuatmang** em tận dụng lại dựa trên cấu trúc bài code thầy Thuận dùng để giảng các loại sort trong buổi 1, 2 của học phần lý thuyết Cấu trúc dữ liệu giải thuật. Nhận con trỏ pa và độ dài len. Dùng vòng lặp truy xuất từng phần tử bằng pointer arithmetic $*(pa + i)$ thay vì $a[i]$, in cách nhau bằng dấu cách. Hàm được gọi trong main để in trạng thái sau khi đã sắp xếp.
2. Hàm **bubblesort**: Hai vòng lặp lồng nhau, i từ 0 đến n-2, j từ i+1 đến n-1. Mỗi lần $*(pa+i) > *(pa+j)$ thì hoán đổi hai phần tử qua biến temp (chọn Bubble Sort cho bài 1 vì thuật toán này trực quan, giúp dễ dàng làm quen và theo dõi từng bước hoán đổi bằng con trỏ trên tập dữ liệu nhỏ)
3. Hàm **docfile** và **ghifile** em áp dụng các ví dụ trong file bài học của bài 1 thực hành. **docfile** mở file "rt", đọc từng số bằng fscanf vào $*(pa + count)$, trả về số phần tử đọc được (đầu file em có giới hạn độ dài của mảng là 100 nên bổ sung (so với code mẫu) thêm điều kiện $count < N$ để tránh vượt ra khỏi mảng gây lỗi. **ghifile** mở file "wt", ghi từng phần tử bằng fprintf.
4. Hàm main có thứ tự chạy như sau:



Mảng $a[N]$ khai báo tĩnh với $N=100$, kích thước thực tế do docfile trả về qua biến n. Ví dụ có file input chứa mảng sau đây (giữa các số nguyên có dấu cách để tránh cho code hiểu lầm mảng chứa 1 số):



Kết quả:

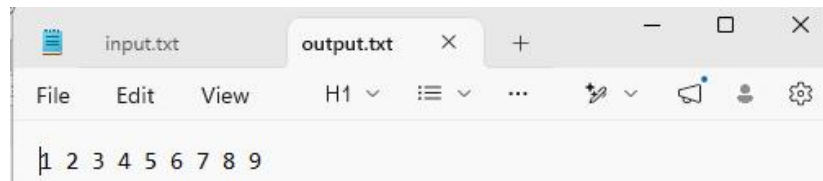
Sau khi chạy chương trình theo thứ tự trên sẽ có kết quả như hình:

```

[Running] cd "d:\daihoc\nam2\HK3\THDSA\Tuan1\" && gcc bai1.c -o bai1 && "d:\daihoc\nam2\HK3\THDSA\Tuan1\"bai1
mang truoc khi sap xep:6 7 9 1 3 5 4 2 8
mang sau khi sap xep:1 2 3 4 5 6 7 8 9

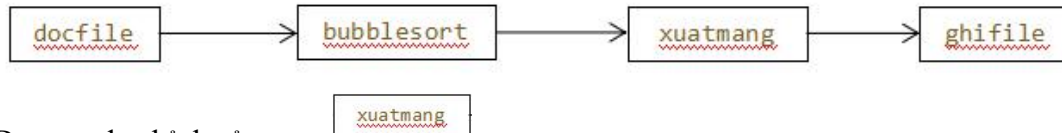
```

Sau đó chương trình dùng hàm ghifile để lưu vào file output.txt có kết quả như hình dưới đây:



Để minh họa rõ hơn cho chương trình:

Em có gọi hàm `xuatmang` trong vòng lặp `bubblesort` để mỗi lần hoán đổi in ra một dòng trạng thái mảng để quan sát từng bước cũng như cách BubbleSort hoạt động:



Đoạn code chỉnh sửa:

```
...
void bubblesort(int *pa, int n) {
    int temp;
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (*(pa + i) > *(pa + j)) {
                temp = *(pa + i);
                *(pa + i) = *(pa + j);
                *(pa + j) = temp;
            }
            xuatmang(pa, n); //gọi hàm xuatmang bổ sung
        }
    }
}
int docfile(char *tenFile, int *pa) {
    ...
}
```

Sau khi chương trình chạy sẽ có kết quả như dưới đây:

Vòng $i=0$
giữ $a[0]$ là
nhỏ nhất

mang trước khi sắp xếp: 6 7 9 1 3 5 4 2 8

[6,7,9,1,3,5,4,2,8] ban đầu $i=0, j=3: 6>1 \rightarrow$ đổi $6 \leftrightarrow 1$
[1,7,9,6,3,5,4,2,8] $\leftarrow$ dòng 1 $i=0, j=4: 1<3$
 $\rightarrow$ bỏ qua ... tất cả j còn lại đều lớn hơn 1 $\rightarrow$ không đổi

Vòng $i=1$
giữ $a[1]$ nhỏ
nhất từ
index 1 trở
đi

[1,7,9,6,3,5,4,2,8]
 $j=1, j=3: 7>6 \rightarrow$ đổi $7 \leftrightarrow 6 \rightarrow$ [1,6,9,7,3,5,4,2,8]
 $j=1, j=4: 6>3 \rightarrow$ đổi $6 \leftrightarrow 3 \rightarrow$ [1,3,9,7,6,5,4,2,8]
 $j=1, j=7: 3>2 \rightarrow$ đổi $3 \leftrightarrow 2 \rightarrow$ [1,2,9,7,6,5,4,3,8]

Vòng $i=2$

[1,2,9,7,6,5,4,3,8]
 $j=2, j=3: 9>7 \rightarrow$ đổi $9 \leftrightarrow 7 \rightarrow$ [1,2,7,9,6,5,4,3,8]
 $j=2, j=4: 7>6 \rightarrow$ đổi $7 \leftrightarrow 6 \rightarrow$ [1,2,6,9,7,5,4,3,8]
 $j=2, j=5: 6>5 \rightarrow$ đổi $6 \leftrightarrow 5 \rightarrow$ [1,2,5,9,7,6,4,3,8]
 $j=2, j=6: 5>4 \rightarrow$ đổi $5 \leftrightarrow 4 \rightarrow$ [1,2,4,9,7,6,5,3,8]
 $j=2, j=7: 4>3 \rightarrow$ đổi $4 \leftrightarrow 3 \rightarrow$ [1,2,3,9,7,6,5,4,8]

Vòng $i=3$

[1,2,3,9,7,6,5,4,8]
 $j=3, j=4: 9>7 \rightarrow$ đổi $9 \leftrightarrow 7 \rightarrow$ [1,2,3,7,9,6,5,4,8]
 $j=3, j=5: 7>6 \rightarrow$ đổi $7 \leftrightarrow 6 \rightarrow$ [1,2,3,6,9,7,5,4,8]
 $j=3, j=6: 6>5 \rightarrow$ đổi $6 \leftrightarrow 5 \rightarrow$ [1,2,3,5,9,7,6,4,8]
 $j=3, j=7: 5>4 \rightarrow$ đổi $5 \leftrightarrow 4 \rightarrow$ [1,2,3,4,9,7,6,5,8]

Vòng $i=4$

[1,2,3,4,9,7,6,5,8]
 $j=4, j=5: 9>7 \rightarrow$ đổi $9 \leftrightarrow 7 \rightarrow$ [1,2,3,4,7,9,6,5,8]
 $j=4, j=6: 7>6 \rightarrow$ đổi $7 \leftrightarrow 6 \rightarrow$ [1,2,3,4,6,9,7,5,8]
 $j=4, j=7: 6>5 \rightarrow$ đổi $6 \leftrightarrow 5 \rightarrow$ [1,2,3,4,5,9,7,6,8]

Vòng $i=5$

[1,2,3,4,5,9,7,6,8]
 $j=5, j=6: 9>7 \rightarrow$ đổi $9 \leftrightarrow 7 \rightarrow$ [1,2,3,4,5,7,9,6,8]
 $j=5, j=7: 7>6 \rightarrow$ đổi $7 \leftrightarrow 6 \rightarrow$ [1,2,3,4,5,6,9,7,8]

Vòng $i=6$

[1,2,3,4,5,6,9,7,8]
 $j=6, j=7: 9>7 \rightarrow$ đổi $9 \leftrightarrow 7 \rightarrow$ [1,2,3,4,5,6,7,9,8]

Vòng $i=7$

[1,2,3,4,5,6,7,8,9]
 $j=7, j=8: 9>8 \rightarrow$ đổi $9 \leftrightarrow 8 \rightarrow$ [1,2,3,4,5,6,7,8,9]

mang sau khi sắp xếp: 1 2 3 4 5 6 7 8 9

Kết quả trong file output.txt vẫn giống như bên trên.

4. Thực hiện:

- Nhập vào một mảng số nguyên từ bàn phím.
- Tính tổng các phần tử trong mảng bằng cách sử dụng đệ quy.
- Sắp xếp mảng theo thứ tự tăng dần bằng cách sử dụng thuật toán Quick Sort và con trỏ.
- In ra mảng đã được sắp xếp.

```
#include <stdio.h>
#include <stdlib.h>
#define N 8
void xuatmang(int *pa, int len) {
    for (int i = 0; i < len; i++)
        printf("%d ", *(pa + i));
    printf("\n");
}
void nhapmang(int *pa, int len) {
    for (int i = 0; i < len; i++) {
        printf("nhap a[%d]: ", i);
        scanf("%d", (pa + i));
    }
}
int tongmang(int *pa, int n) {
    if (n == 0)
        return 0;
    return *(pa + n - 1) + tongmang(pa, n - 1);
}
int phanhoach(int *pa, int trai, int phai) {
    int pivot = *(pa + phai);
    int i = trai - 1, temp;

    for (int j = trai; j < phai; j++) {
        if (*(pa + j) <= pivot) {
            i++;
            temp = *(pa + i);
            *(pa + i) = *(pa + j);
            *(pa + j) = temp;
        }
    }
    temp = *(pa + i + 1);
    *(pa + i + 1) = *(pa + phai);
    *(pa + phai) = temp;

    return i + 1;
}
void quicksort(int *pa, int trai, int phai) {
    if (trai >= phai)
        return;
    int vitripivot = phanhoach(pa, trai, phai);
    quicksort(pa, trai, vitripivot - 1);
```

```

    quicksort(pa, vitripivot + 1, phai);
}
int main() {
    int a[N];
    nhapmang(a, N);
    printf("mang truoc khi sap xep: ");
    xuatmang(a, N);
    printf("tong cac phan tu: %d\n", tongmang(a, N));
    quicksort(a, 0, N - 1);
    printf("mang sau khi sap xep: ");
    xuatmang(a, N);

    return 0;
}

```

Giải thích cách hoạt động của code:

1. Hàm **xuatmang** đã đề cập ở bài 1. Hàm **nhapmang** cũng được tham khảo từ cấu trúc bài code thầy Thuận dùng để giảng các loại sort trong buổi 1, 2 của học phần lý thuyết CTDLGT. Nhận con trỏ pa và độ dài len. Dùng vòng lặp for từ 0 đến len-1, mỗi lần in nhãn “nhap a[i]: “ rồi dùng scanf (“%d”, pa+i) để đọc giá trị thẳng vào địa chỉ phần tử thứ i. Hàm được gọi đầu tiên trong main để thu thập dữ liệu từ bàn phím.

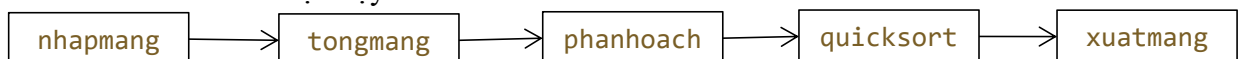
2. Hàm **tongmang**: Dùng đệ quy để tính tổng các phần tử trong mảng. Điều kiện dừng là khi kích thước mảng $n == 0$ trả về 0. Ngược lại, trả về $*(pa + n - 1) + tongmang(pa, n - 1)$ (lấy phần tử cuối cùng cộng với tổng n-1 phần tử trước đó).

$tongmang(a, 8) = a[7] + tongmang(a, 7) = a[7] + a[6] + \dots + a[0] + tongmang(a, 0)$
 $= a[7] + a[6] + \dots + a[0] + 0$

3. Hàm **phanhoach**: Chọn phần tử cuối cùng của đoạn đang xét $*(pa+phai)$ làm pivot. Dùng biến $i = \text{trai} - 1$ làm giới hạn: mọi phần tử $*(pa+j) \leq \text{pivot}$ được hoán đổi vào vùng bên trái ($i++$ rồi swap $pa+i$ với $pa+j$). Sau vòng lặp, đặt pivot về đúng vị trí $i+1$ bằng cách swap với $*(pa+phai)$. Trả về $i+1$ (vị trí của pivot sau phân hoạch)

4. Hàm **quicksort**: Đệ quy theo cơ chế chia để trị. Điều kiện dừng là $\text{trai} \geq \text{phai}$ (đoạn có 0 hoặc 1 phần tử, đã có thứ tự). Gọi **phanhoach** để xác định vị trí pivot (vitripivot) sau đó đệ quy độc lập 2 nửa ($\text{trai}, \text{vitripivot}-1$) và ($\text{vitripivot}+1, \text{phai}$). Vì pivot đã nằm đúng vị trí, hai nửa không phụ thuộc nhau. (Quick Sort được sử dụng nhằm đồng nhất với phương pháp đệ quy của hàm tính tổng, đồng thời mang lại hiệu suất, tối ưu bộ nhớ)

5. Hàm main có thứ tự chạy như sau



Mảng $a[N]$ khai báo tĩnh với $N=8$. Ví dụ nhập mảng: 1 3 5 7 2 4 6 8 (nhập lần lượt từng phần tử qua bàn phím).

Kết quả

Sau khi chạy chương trình theo thứ tự trên sẽ có kết quả như hình:

```

PS D:\daihoc\nam2\HK3\THDSA\Tuan1> .\bai4.exe
nhap a[0]: 1
nhap a[1]: 3
nhap a[2]: 5
nhap a[3]: 7
nhap a[4]: 2
nhap a[5]: 4
nhap a[6]: 6
nhap a[7]: 8
mang truoc khi sap xep: 1 3 5 7 2 4 6 8
tong cac phan tu: 36
mang sau khi sap xep: 1 2 3 4 5 6 7 8

```

Để minh họa rõ hơn cho chương trình:

Như ở bài 1, em đã bổ sung các lệnh printf trong quicksort để in ra đoạn đang xử lý, pivot được chọn, vị trí đặt pivot và trạng thái mảng sau mỗi lần phân hoạch giúp quan sát từng bước QuickSort hoạt động:

```

...
void quicksort(int *pa, int trai, int phai) {
    if (trai >= phai)
        return;
    printf("\ndang xu ly khoang tu index %d den %d \n", trai, phai);
    int vitripivot = phanhoach(pa, trai, phai);
    printf("pivot duoc chon la %d dat tai index %d\n", *(pa + vitripivot),
        vitripivot);
    xuatmang(pa, N);
    quicksort(pa, trai, vitripivot - 1);
    quicksort(pa, vitripivot + 1, phai);
}
...

```

Sau khi chương trình đã bổ sung chạy sẽ có kết quả như dưới đây:

```

PS D:\daihoc\nam2\HK3\THDSA\Tuan1> .\bai4.exe
nhap a[0]: 1
nhap a[1]: 3
nhap a[2]: 5
nhap a[3]: 7
nhap a[4]: 2
nhap a[5]: 4
nhap a[6]: 6
nhap a[7]: 8
mang truoc khi sap xep: 1 3 5 7 2 4 6 8
tong cac phan tu: 36

```

Màn hình nhập lần lượt 8 phần tử: 1 3 5 7 2 4 6 8. In mảng trước sắp xếp và tổng các phần tử ($1+3+5+7+2+4+6+8 = 36$).

dang xu ly khoang tu index 0 den 7
pivot duoc chon la 8 dat tai index 7
1 3 5 7 2 4 6 8

dang xu ly khoang tu index 0 den 6
pivot duoc chon la 6 dat tai index 5
1 3 5 2 4 6 7 8

dang xu ly khoang tu index 0 den 4
pivot duoc chon la 4 dat tai index 3
1 3 2 4 5 6 7 8

dang xu ly khoang tu index 0 den 2
pivot duoc chon la 2 dat tai index 1
1 2 3 4 5 6 7 8

mang sau khi sap xep: 1 2 3 4 5 6 7 8

Giai đoạn xử lý	Pivot chọn	Vị trí pivot	Trạng thái mảng sau phân hoạch
index 0→7	8 (index 7)	index 7	1 3 5 7 2 4 6 8
index 0→6	6 (index 5)	index 5	1 3 5 2 4 6 7 8
index 0→4	4 (index 3)	index 3	1 3 2 4 5 6 7 8
index 0→2	2 (index 1)	index 1	1 2 3 4 5 6 7 8

Các bước cụ thể:

Đầu tiên, nhập mảng [1, 3, 5, 7, 2, 4, 6,8]. Hàm đệ quy tính tổng hoạt động trả về kết quả là 36.

Lần chia 1:

Xử lý toàn bộ mảng từ index 0 đến 7. Pivot được chọn là phần tử cuối (giá trị 8). Do 8 là giá trị lớn nhất, vị trí của pivot giữ nguyên ở index 7.

Lần chia 2:

Xét nửa trái từ index 0 đến 6 [1, 3, 5, 7, 2, 4, 6]. Pivot là 6. Các giá trị nhỏ hơn 6 bị dồn về bên trái, đổi chỗ 7 và 2, 4 giữ nguyên. Pivot được chèn vào giữa, đặt tại index 5.

Lần chia 3:

Xét tiếp nửa trái từ index 0 đến 4 [1, 3, 5, 2, 4]. Pivot là 4. Hoán đổi đưa 5 ra sau và chốt 4 vào index 3.

Lần chia 4:

Xét index từ 0 đến 2 [1, 3, 2]. Pivot là 2. Đưa 3 ra sau và chèn 2 lên vị trí index 1.

Các mảng con lúc này chỉ còn 1 phần tử nên đệ quy dừng lại. Hàm xuất mảng được gọi lại trong main để in kết quả sau khi sắp xếp.